

Preface

Before You Vibe-Code

When building becomes easier than deciding

In early 2025, a new way of creating software entered the technology conversation: vibe coding. The idea was simple and, in many ways, revolutionary. Instead of writing every line of code by hand, a person could describe what they wanted in natural language and let AI generate much of the implementation. The interaction between human and machine became less like traditional programming and more like a conversation: describe an intention, look at the result, refine the request, run the software, inspect what happened, and continue.

For people who had never considered themselves programmers, this opened a remarkable new possibility. An idea could become a working application without first mastering a programming language, a framework, a database system, or a modern software stack. The barrier between having an idea and building software was suddenly much lower.

That is an extraordinary opportunity. But it also creates a new problem.

The easier software becomes to build, the more important it becomes to decide what is actually worth building.

The discussion around AI-assisted software development has understandably focused on the implementation side: How can AI generate code? How can coding agents work with repositories? How can prompts, specifications, tests, and tools make AI-generated software more reliable?

These are important questions. But there is another question that comes first: What should we ask the AI to build in the first place? That question is at the heart of this book.

From an Idea to a product decision

Imagine someone saying, “I have an idea for an app for book lovers. It should scan books, create a personal library, and recommend what to read next.”

With today’s AI coding tools, it may not take long before something resembling that idea is running on a screen.

But what has actually been decided?

Who exactly is the user? What problem does this person have? How important is that problem? How is it solved today? Why would someone change their existing behavior? What outcome does the user really want? Which part of the idea matters most? What assumptions are we making? What needs to be tested? What should the first version deliberately not do? How will we know whether the product is useful? And, if this is intended to become a business, who will pay for it and why?

None of these questions is primarily a coding question. They are product questions. And they cannot be answered simply by generating more code.

This distinction matters because a working application is not necessarily a useful product. An application can be technically impressive, beautifully designed, and full of functionality while solving a problem that is not important enough, for users who do not need it, in a market that does not support it.

A working app is evidence that something can be built. It is not evidence that it should be built.

The problem is not a lack of ideas

We have never had a shortage of ideas. What has traditionally been expensive is turning an uncertain idea into something that can be tested in the real world.

Over time, product development has drawn on a number of disciplines to address different parts of this challenge: product management, customer development, user research, Lean Startup, requirements engineering, UX design, business modeling, and many others. Each addresses part of the problem.

But AI changes the context in which these disciplines are applied.

A person can now move from an informal thought—"I think there should be an easier way to ..."—to a functioning software prototype remarkably quickly. That changes the order in which things can happen.

Traditionally, turning an idea into working software required substantial implementation effort. Before something could be built, someone had to understand the technology well enough to build it. AI-assisted development can substantially reduce some of that implementation friction.

This is why the next challenge is not simply to build faster. It is to think clearly enough before building.

The goal is not to replace creativity with process. Nor is it to turn every idea into a lengthy requirements document before anything can happen. The goal is to use AI to help move from an uncertain idea to a coherent, focused, testable, and buildable MVP.

Rethinking the MVP

This brings us to another term that is often misunderstood: Minimum Viable Product.

MVP does not simply mean "the product with the fewest features." The influential thinking behind the term points in a different direction. Frank Robinson framed the MVP around balancing product return and risk.[1] Eric Ries emphasized validated learning with the least effort.[2] Steve Blank emphasized using an MVP to test hypotheses and learn from customers.[3]

The common idea is that an MVP exists for a reason. It should help us learn something important. It should contain enough of the product experience for the relevant hypothesis to be tested meaningfully. And the evidence obtained should inform the next product decision.

A more useful way to think about an MVP is as the smallest coherent product that is valuable enough to matter, viable enough to make sense, and buildable enough to implement—and that can generate meaningful evidence for an important product decision. “Minimum” concerns scope; “valuable” concerns the value delivered to the intended user; “viable” concerns whether the product can support a plausible value or business case; and “buildable” concerns whether it can realistically be implemented and operated. The MVP is also an instrument for learning: it should help the team reduce important uncertainty and make the next product decision.

That distinction is crucial. If we simply remove features from a product until very little remains, we may have achieved “minimum” without achieving “viable.” Conversely, if we keep adding functionality until the product feels complete, we may have created something far larger and more expensive than the evidence we actually need.

The question is therefore not, “How many features can we fit into the MVP?” It is, “What is the smallest coherent product experience that allows us to test what matters?” That is a very different starting point.

Before Vibe Coding comes Vibe Specification

In this book, we use the term “Vibe Specification” for the AI-assisted exploration that happens before a precise MVP Specification is produced.

The term is intentionally connected to the world of vibe coding. But the direction is reversed.

Vibe coding can be summarized as: “I have an idea of what I want to build. Help me build it.”

Vibe Specification starts one step earlier: “I have an idea of what I might build. Help me figure out what I should actually build.”

The difference may sound small. It is not.

Vibe Specification is the process of exploring an initially vague product idea through an AI-assisted conversation. The purpose is to clarify the problem, understand the intended user, identify the value, expose assumptions, determine what needs to be validated, reduce unnecessary scope, and progressively turn the idea into a precise MVP specification.

The input can be informal. The process can be conversational. The human does not need to begin with a complete product specification. In fact, the starting point may be nothing more than, “I have an idea for an app ...”

The AI should not merely say, “Great idea. Here is your feature list.” It should help ask: Who is this for? What problem are we solving? What happens today? What outcome does the user really want? What are we assuming? Which assumptions matter most? What is the smallest useful scope? What should we leave out? How will we know whether the MVP works?

In other words, AI can become more than a builder. It can become a thinking companion.

The process can be a vibe. The specification cannot.

There is an important paradox here. The starting point can be vague. The conversation can be exploratory. Ideas can change. Assumptions can be challenged. The user can discover that the original idea was not quite the right one.

But the output should become increasingly precise.

The transformation we are looking for is: Vague Idea → Guided Conversation → Structured Thinking → Precise MVP Specification.

That is the fundamental idea behind MVPCompanion framework.

MVPCompanion is designed to provide a systematic path from an uncertain product idea to a coherent, buildable, and learnable MVP. The method progressively clarifies the product rather than assuming that all important decisions are known at the beginning. Its stages move through questions concerning the problem, users, value, product, user goals, scope, experience, requirements, learning, technical boundaries, and the final MVP boundary.

The objective is not documentation for its own sake. The objective is better product decisions before implementation.

And that creates a simple sequence: Idea → Vibe Specification → MVP Specification → Vibe Coding → Working MVP → Validation → Learning.

Why this book?

MVPCompanion was developed around a recurring challenge: deciding what should be built is often as important as building it.

We need to become better at moving from an idea to a problem, from a problem to a user, from a user to an outcome, from an outcome to a product capability, from a capability to a requirement, from a requirement to an acceptance criterion, and ultimately from an acceptance criterion to measurable evidence.

This traceability is one of the principles that runs through MVPCompanion. A product decision should not appear out of nowhere. It should have a reason, and that reason should remain visible as the idea becomes more concrete.

A different way to use AI

Perhaps the most important idea in this book is therefore not about coding at all. It is about how we think with AI.

AI can generate code. It can generate screens. It can generate database schemas, APIs, tests, and documentation. But before asking AI to generate all of these things, we can ask it to help us think.

We can use AI to challenge assumptions, expose ambiguity, reduce scope, and ask questions we might otherwise skip. We can also use it to transform an idea that exists mainly in someone's head into a product specification that another human—or an AI coding agent—can work with.

A few pages from now, we will start with an idea. It may sound reasonable, even exciting. But we will first ask what lies underneath it: What problem are we solving? For whom? Why does it matter? What would the user actually need? What should the first version do—and what should it deliberately not do? What would make the MVP worth testing, and what evidence would tell us what to do next?

So if AI has made you faster at building software, this book is not here to slow you down. It is here to help you pause at the right moment.

Before the first prompt to the coding agent. Before the first generated screen. Before the first feature. Before the code.

Ask the more fundamental question: What are we actually trying to build—and why?

Before you vibe-code, vibe-specify.

Sources

[1] Frank Robinson, “MVP: How It Works,” SyncDev, 2008. Robinson framed the MVP around balancing product return and risk.

[2] Eric Ries, “What Is an MVP?”, Lean Startup Co. Ries defines the MVP in terms of collecting the maximum amount of validated learning about customers with the least effort.

[3] Steve Blank, “Why Build, Measure, Learn ... the Minimal Viable Product,” SteveBlank.com, 2015. Blank describes an MVP as the simplest thing that can be shown to customers to maximize learning at that point in time.

Source treatment: These references acknowledge influential perspectives on the MVP concept. The MVPCompanion definition developed in this book is its own methodological formulation and is not a quotation or synthesis attributed to any one of these authors.